

Enjambre: Simulación de Comportamiento Colectivo con Boids, Renderizado Instanciado y Shaders Propios en Three.js

Javier Arturo Cirilo Vargas
Universidad Autónoma
Metropolitana – Cuajimalpa
Ciudad de México, México

Julio César Aguirre Cresencio
Universidad Autónoma
Metropolitana – Cuajimalpa
Ciudad de México, México

Camila Anaid Cruz Rodríguez
Universidad Autónoma
Metropolitana – Cuajimalpa
Ciudad de México, México

Abstract

Este reporte documenta el proyecto final *Enjambre*, una simulación interactiva de comportamiento colectivo tipo *boids* implementada en JavaScript con Three.js y WebGL2. El sistema integra las entregas previas del curso (escena base, geometría del enjambre, shaders propios e instanciado con animación por textura) en una única aplicación autocontenida ejecutable en cualquier navegador. Se describe la arquitectura del proyecto, el modelo de comportamiento de Craig Reynolds (separación, alineación y cohesión) acelerado mediante un *spatial hash grid*, la técnica de *Vertex Animation Texture* (VAT) combinada con *InstancedMesh* para animar cientos de agentes en dos llamadas de dibujo, y los shaders propios de *toon shading* y deformación de vértices. Se justifican las decisiones técnicas tomadas frente a alternativas posibles y se discuten los resultados de rendimiento obtenidos.

1 Introducción

El comportamiento de enjambre —bandadas de aves, cardúmenes de peces, colonias de insectos— es uno de los ejemplos clásicos de *vida artificial* y gráficos por computadora, popularizado por el modelo de *boids* de Craig Reynolds [1]. Este proyecto retoma dicho modelo como eje central de un sistema de renderizado en tiempo real que combina simulación física, geometría procedural, animación eficiente en GPU y *shading* no fotorrealista.

El proyecto se desarrolló de forma incremental a lo largo del curso: una primera entrega estableció la escena base y la iluminación; una segunda introdujo el enjambre con un modelo de alineación simplificado; entregas posteriores añadieron shaders propios e instanciado con animación horneada en textura. El entregable final, *Enjambre*, integra y reemplaza estas piezas en un solo archivo `index.html` sin dependencias de compilación, con el objetivo de que el comportamiento colectivo, antes aproximado, se resolviera mediante el algoritmo completo de Reynolds y escale a varios cientos de agentes manteniendo una tasa de cuadros interactiva.

El objetivo de este documento es explicar qué se construyó, qué herramientas y técnicas se emplearon, y por qué se tomaron las decisiones técnicas específicas descritas en las secciones siguientes.

2 Descripción General del Proyecto

Enjambre simula entre 50 y 500 agentes con forma de pez/ave que se desplazan dentro de una caja de límites, evitando colisionarse entre sí, alineando su velocidad con la de sus vecinos y agrupándose alrededor del centro de masa local. El

usuario puede ajustar en vivo el número de agentes, el radio de percepción, los pesos de cada regla de comportamiento, la velocidad máxima, el shader activo y parámetros visuales como el número de bandas del *toon shading* o la amplitud de una deformación tipo ola. Un panel de *heads-up display* (HUD) reporta cuadros por segundo, número de llamadas de dibujo y triángulos renderizados, lo que permite observar directamente el efecto de cada decisión de optimización.

La aplicación no requiere `npm install` ni proceso de compilación: toda la lógica —simulación, geometría, shaders GLSL y control de interfaz— vive en un único archivo HTML que carga Three.js y se ejecuta sobre WebGL2. Por restricciones de CORS al muestrear texturas de datos, el proyecto debe servirse mediante un servidor HTTP local (por ejemplo `python -m http.server`) en lugar de abrirse directamente con el protocolo `file://`.

3 Decisiones Técnicas y Arquitectura

3.1 Modelo de comportamiento: boids con aceleración espacial

La decisión más importante del proyecto fue reemplazar el esquema de alineación simplificado de la entrega anterior —donde cada agente simplemente interpolaba su velocidad hacia el promedio del enjambre— por el modelo completo de Reynolds con sus tres reglas de *steering* [1]: separación (alejarse de vecinos demasiado cercanos, ponderada por la inversa de la distancia), alineación (igualar la velocidad promedio de los vecinos) y cohesión (dirigirse hacia el centro de masa local). Cada fuerza se calcula por separado, se limita a una magnitud máxima (*steering force clamp*) y se combina con pesos ajustables desde la interfaz, lo que permite observar en vivo cómo cada regla afecta la coherencia visual de la bandada.

Calcular estas tres reglas ingenuamente requiere comparar cada agente contra todos los demás, un costo $O(n^2)$ que se vuelve prohibitivo al escalar a cientos de agentes por cuadro. Se optó por construir, en cada cuadro, una tabla *hash* espacial (`Map` de JavaScript) que indexa a cada agente por la celda discreta que ocupa, con un tamaño de celda igual al radio de percepción. Al buscar vecinos, cada agente solo consulta las 27 celdas contiguas (vecindad $3 \times 3 \times 3$) en vez de recorrer el enjambre completo, lo que reduce el costo promedio a $O(n)$ y permite escalar de 200 a 500 agentes sin penalización severa de cuadros por segundo. Se prefirió esta estructura sobre alternativas como *k-d trees* u *octrees* porque su reconstrucción por cuadro es trivial (no requiere balanceo) y su costo de inserción/consulta es adecuado para el rango

de agentes solicitado; para escalar a miles de agentes, la ruta natural sería mover el cálculo de *flocking* a un *compute shader* en GPU, alternativa que se descartó por complejidad frente al beneficio para el alcance del proyecto.

Adicionalmente, en lugar de teletransportar a los agentes que llegan al borde de la caja de límites (*wrap-around*), se implementó un “cerco invisible”: una fuerza de dirección suave que empuja a los agentes de regreso al centro conforme se acercan al borde. Esta decisión favorece la coherencia visual del enjambre como bandada contenida, evitando el efecto de partículas que desaparecen y reaparecen abruptamente.

3.2 Renderizado eficiente: InstancedMesh y Vertex Animation Texture

Dibujar cada agente como una malla independiente implicaría una llamada de dibujo por agente, lo cual es costoso incluso para cientos de instancias. Se utilizó `THREE.InstancedMesh`, que permite dibujar todas las copias de una misma geometría en una sola llamada, actualizando únicamente una matriz de transformación por instancia en cada cuadro. El agente y su contorno (*outline*) se dibujan como dos `InstancedMesh` independientes, de modo que el enjambre completo —sin importar si son 50 o 500 agentes— se renderiza en solo dos *draw calls*.

Para animar el “nado” de cada agente sin recalcular geometría en la CPU, se adoptó la técnica de *Vertex Animation Texture* (VAT): la animación de ondulación se hornea de antemano en una `DataTexture RGBA Float32`, donde cada columna corresponde a un vértice de la malla y cada fila a un cuadro de animación (64 cuadros en total); cada texel codifica el desplazamiento ($\Delta x, \Delta y, \Delta z$) y un factor de compresión (*squash*) respecto a la posición de reposo. El *vertex shader* muestrea esta textura con `gl.VertexID` para reconstruir la posición animada de cada vértice en la GPU, evitando actualizar geometría por CPU en cada cuadro. Esta técnica, ampliamente utilizada en motores de videojuegos para animar grandes cantidades de instancias con bajo costo de CPU [2, 3], se eligió sobre animación esquelética (*skinning*) porque no requiere jerarquía de huesos ni cálculo de matrices por hueso en tiempo real, y se integra de forma natural con `InstancedMesh`.

Para que los cientos de agentes no se vean sincronizados como un único organismo, cada instancia recibe un atributo `aTimeOffset` aleatorio que desfasa su lectura de la VAT, de modo que cada uno reproduce la misma animación horneada en una fase distinta. La orientación de cada instancia, por su parte, no proviene de la animación sino que se recalcula cada cuadro con `Object3D.lookAt()` apuntando en la dirección real de la velocidad resultante del cálculo de *flocking*, de forma que el giro de cada agente refleja su propio comportamiento y no un patrón preestablecido.

La geometría del agente es una malla de bajo poligonaje construida a mano (cuerpo tipo bipirámide, aletas y cola en horquilla, aproximadamente 39 vértices) en lugar del icosaedro usado en entregas previas. Al construirse ya “no indexada” (sin buffer de índices, con vértices duplicados por triángulo),

encaja directamente en el esquema de `gl.VertexID` que requiere la VAT sin pasos de conversión adicionales, y su bajo conteo de vértices mantiene barato el costo de vértices incluso con 500 instancias simultáneas.

3.3 Shaders propios

Se implementaron dos modos de *shading* intercambiables en tiempo real, ambos escritos como `THREE.ShaderMaterial` con GLSL propio en lugar de materiales predefinidos de Three.js, para cumplir el requisito de shaders desarrollados por el equipo:

- **Toon / Cel-shading:** la componente difusa de la iluminación se cuantiza con `floor(diff * bandas)` / `bandas`, produciendo bandas de luz discretas en vez de un degradado continuo, y se añade un resalte especular discreto mediante `step()`. El número de bandas es ajustable desde la interfaz.
- **Vertex Wobble:** una deformación senoidal de la forma $A \sin(\text{freq} \cdot y - \text{speed} \cdot t)$ se aplica sobre la posición ya animada por la VAT, junto con un gradiente de color propio, generando un efecto de ondulación adicional sobre el nado base.

El contorno (*outline*) de cada agente se logra con la técnica de *back-face inflate*: un segundo `InstancedMesh` con la misma geometría, renderizado con `side: THREE.BackSide` y los vértices desplazados ligeramente a lo largo de su normal, de modo que solo se ve el borde trasero inflado detrás de la malla principal. Esta técnica se prefirió sobre un *post-processing pass* de detección de bordes por ser más económica y por integrarse directamente con el esquema de instanciado ya existente, sin pases adicionales de render-to-texture.

3.4 Escena e iluminación

La escena reutiliza la iluminación de la primera entrega —una `PointLight` principal, una `AmbientLight` fría y una `HemisphereLight`— para que el enjambre se perciba integrado en el mismo entorno visual sin importar el modo de shader activo. Se añadió niebla exponencial (`FogExp2`) para dar sensación de profundidad, un piso tipo *grid*, un campo de estrellas de fondo y una caja de límites opcional que hace visible el volumen dentro del cual se contiene el enjambre. La cámara orbita automáticamente alrededor de la escena, aunque puede desactivarse desde la interfaz.

4 Resultados y Discusión de Rendimiento

Con 200 agentes —el caso base solicitado— el costo dominante deja de ser el renderizado, gracias a las dos llamadas de dibujo logradas con `InstancedMesh`, y pasa a ser el cálculo de *flocking* en CPU. Gracias al *spatial hash grid*, dicho cálculo se mantiene cercano a $O(n)$, lo que permite subir hasta 500 agentes sin una caída drástica de cuadros por segundo en hardware convencional. El HUD integrado permite comparar visualmente el efecto de habilitar o deshabilitar distintos parámetros (radio de percepción, pesos de cada regla) sobre la cohesión del enjambre y sobre el desempeño.

La principal limitación identificada es que, más allá de unos pocos miles de agentes, el cálculo de *flocking* en CPU volvería a ser el cuello de botella, incluso con la aceleración espacial. La extensión natural sería mover dicho cálculo a un *compute* o *vertex shader* en GPU, usando texturas de posición y velocidad en lugar de arreglos de JavaScript, tal como se hace en implementaciones GPGPU de boids a gran escala. Para el rango de agentes pedido en el proyecto (200, hasta 500), sin embargo, el enfoque híbrido CPU+grid resultó suficiente y considerablemente más simple de depurar y ajustar en vivo que una implementación GPGPU completa.

5 Conclusiones

El proyecto *Enjambre* integra en una sola aplicación los conceptos trabajados a lo largo del curso: iluminación y composición de escena, geometría procedural, *shaders* propios, e instanciado eficiente con animación horneada en textura, todo ello puesto al servicio de un modelo de comportamiento colectivo fiel al algoritmo original de Reynolds. Las decisiones

técnicas —spatial hashing para el cálculo de vecinos, VAT combinada con **InstancedMesh** para la animación, y shaders GLSL propios para el estilo visual— se tomaron priorizando un balance entre fidelidad al comportamiento de enjambre, rendimiento en tiempo real y simplicidad de depuración, dejando explícitamente documentada la ruta de escalamiento a GPGPU como trabajo futuro si se requirieran miles de agentes simultáneos.

References

- [1] Craig W. Reynolds. 1987. Flocks, Herds, and Schools: A Distributed Behavioral Model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '87)*. ACM, 25–34. doi:10.1145/37401.37406
- [2] Luiz Otavio Vasconcelos. 2020. Texture Animation: Applying Morphing and Vertex Animation Techniques. Wildlife Studios Tech Blog, <https://medium.com/tech-at-wildlife-studios/texture-animation-techniques-1daecb316657>. Consultado en 2026.
- [3] Quentin Warnant. 2023. Vertex Animation Textures. <https://quentinw.com/graphics/vertex-animation-textures/>. Consultado en 2026.